KANN AUDITS

Security Review
For Castr.fun

Prepared For: **Castr.fun**

Date Reported: **August 13, 2026**

Audit Summary

Castr.fun engaged Kann Audits to perform a security review of their bonding curve token launch protocol. From April 4 to April 26, a team of security researchers performed a manual review of the in-scope source code and additionally triaged findings generated by the Kann Labs Security Model. All findings identified during the assessment have been documented in the following report.

Protocol Overiew

castr.fun is a fully on-chain token launchpad built on Base, powered by Bancor-style bonding curves. It lets anyone mint and trade a new token instantly against a virtual liquidity curve, with no upfront pool needed, then automatically graduates it into a locked Aerodrome Slipstream concentrated-liquidity pool once it hits its funding goal. Built for creators, communities, and traders who want fair-launch price discovery with built-in anti-sniping, vesting, and dividend/reward mechanics that keep incentives aligned long after migration.

Security Researchers

parvanj
Uzair Saiyed
anchabadze
yusufthebddev
Luc1jan
theBugLifestyle
shieldrey
hadramaut
Volodymyr
skipper
stranger8732
0xbadal
Count-Sum
0xhalas
dinkras
AlexScherbatyuk
yunohu
ibukun
VCeb
Riceee
theBugLifestyle
Digital Forensic
Shreyash
derastephh
FalseGenius
Pindarev
grayson
victorygod
heeze
0xkujen
Jerry
Fishy
Razkky
Lookman
syedghufranhassan

Scope

Repository: https://github.com/VasilGrozdanov/castr.fun-audit/

Audited Commit: 5ecd99c

Files:

- src/AddressChecker.sol
- src/BancorBondingCurve.sol
- src/BondingCurve.sol
- src/BondingCurveFactory.sol
- src/BondingCurvesStorage.sol
- src/BurnableToken.sol
- src/FactoryManager.sol
- src/FeeAccount.sol
- src/LiquidityLocker.sol
- src/LiquidityMigrator.sol
- src/LockManager.sol
- src/Power.sol
- src/RewardSolver.sol
- src/SoulboundNFT.sol
- src/libraries/NFTDescriptor.sol
- src/libraries/NFTSVG.sol
- src/libraries/StringValidator.sol
- src/libraries/VestingCalculator.sol

Methodology

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms, and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

The auditing process specifically considered:

- Testing against common and uncommon attack vectors
- Ensuring compliance with industry best practices and known smart contract security standards
- Verifying contract logic matches the client's intended behavior
- Cross-referencing patterns with industry-standard implementations
- Manual line-by-line review by experienced auditors
- Access control verification
- Architecture and design analysis to ensure secure component interactions and clearly defined trust boundaries
- State transition and logic review to detect inconsistencies, unexpected behavior, or potential exploitation paths

Risk Classification

- **High** – result in directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
- **Medium** – result in moderate loss of funds or impact multiple users, but only under specific conditions. They are exploitable without privileged access but typically require certain assumptions or edge cases.
- **Low/Info** – issues are non-exploitable findings that do not pose a direct security risk or impact the system's integrity. These issues are typically cosmetic, best-practice deviations, or related to documentation and compliance. While they do not require urgent remediation, they may be addressed to improve code quality and maintainability.

Findings Count

Critical/High	Medium	Low/Info
24	26	37

Findings Summary

ID	Title	Severity
C-01	Permissionless Rogue Clone Attack: BondingCurve.initialize() Accepts Unvalidated Migrator and Factory Addresses, Enabling Total Fund Drain at Graduation	Critical
C-02	LockManager: New lock positions can claim all past rewards and dividends	Critical
C-03	Double-Claim Token Drain via Reentrancy in claimRewards	Critical
C-04	Unguarded migrateLiquidity() Function Causes Loss of User Earnings (CVSS 9.5)	Critical
H-01	Attacker can steal the proxies' assets	High
H-02	collectFeesAndDistribute missing return causes approve(address(0)) revert when lockManager is uninitialized	High
H-03	The Swap Protection Limit Is Calculated Using the Wrong Token and the Wrong Amount	High
H-04	Reward Swaps Use a Price That Can Be Manipulated in the Same Block	High
H-05	performUpkeep/_rewardWinner Re-Includes LP-Fee WETH and Burns LP-Fee Winner Tokens	High
H-06	Permissionless pre-creation of the Slipstream pool permanently DoSes migration	High
H-07	Dust Sent to the Token Contract Can Permanently Brick the First Lock Initialization	High
H-08	Stale Oracle Response Accepted - requestId Never Validated	High
H-09	Hardcoded Sepolia configurations in RewardResolver.sol cause total failure on Base mainnet deployment	High
H-10	Missing Zero-Shares Guard in distributeDividends() and distributeRewards()	High
H-11	Dividend redistribution in case of failure leads to loss of funds.	High
H-12	virtualLiquidity Permanently Drifts Due to Sell Fee Accounting Error	High

ID	Title	Severity
H-13	Cloned bonding curves inherit constructor-initialized Power storage that is never initialized in clone instances	High
H-14	safeTransferFrom bypasses soulbound restrictions and desynchronizes LockPositionsNFT ownership tracking	High
H-15	FeeAccount.onERC721Received accepts arbitrary LockPositionsNFT deposits, permanently stranding a perpetual share of every future reward and dividend distribution	High
H-16	Fee Exemption Mechanism Critically Broken: Any Transferable NFT Enables Arbitrary Fee Bypass	High
H-17	Superlinear Vesting Exponent (1.3) Enables Sybil-Based Vesting Bypass via Pre-Graduation Wallet Splitting	High
H-18	Direct LockManager.lock() call bypasses post-migration vesting lock-floor in one transaction, letting any vested holder liquidate their entire balance during vesting	High
H-19	_transferEtherToFeeAccount() Will Always Revert Because Contract Has No receive() Function	High
H-20	_Soulbound Property Bypassed via 'safeTransferFrom'	High
M-01	BondingCurve drives king-of-the-casts and migration state transitions off address(this).balance, allowing forced-ETH griefing via selfdestruct	Medium
M-02	FeeAccount WETH accumulation double-counts, eventually reverting performUpkeep	Medium
M-03	Reward Upkeep Spends WETH That Was Reserved for Per-Token Fee Distribution	Medium
M-04	LockManager: Last Lock Redistribution Always Panics	Medium
M-05	Missing Slippage Protection During Liquidity Migration Enables Sandwich Attacks	Medium
M-06	BurnableToken.burn() under-clamps the vesting accumulator, letting any post-launch locker brick their own transfers with a 1-wei burn	Medium
M-07	BondingCurve._checkIsMigrating passes stale tokensForLiquidity to setMigrated after _manageSpareTokens burn	Medium
M-08	kingsOfTheCasts Array Corruption When currentKingOfTheCasts Migrates	Medium
M-09	basedNFTs[0] Only Ever Checked - Multi-NFT Fee Waiver Support Is Broken	Medium
M-10	Bonding Curve Permanent DoS Due to Underflow in BondingCurve::_extendHolderTransfersLock	Medium

ID	Title	Severity
M-11	VestingCalculator Reverts on Zero totalVestedSupply	Medium
M-12	checkUpkeep Does Not Validate ETH Balance, Leading to Wasted LINK	Medium
M-13	Fee Percentages in FeeAccount Not Validated Against 100% Cap	Medium
M-14	Incorrect reward calculation causes inversed reward calculation for winners in performUpkeep	Medium
M-15	collectRewardsAndDividends Has No ownerOf Guard	Medium
M-16	State Machine Deadlock in CALCULATING State in RewardResolver.sol	Medium
M-17	FeeAccount.collectFeesAndDistribute Lets the Freshly Minted Dev Lock Share in the Same Reward Distribution	Medium
M-18	lock() First Caller Silently Receives Undocumented Boost - First-Mover Advantage	Medium
M-19	Deterministic Rarity Calculation Is Predictable and Gameable	Medium
M-20	Unconstrained Handle Input on Mint Enables JSON Injection and Storage Bloat	Medium
M-21	tokenURI Does Not Validate Token Existence	Medium
M-22	Owner Can Arbitrarily Mutate Global Metadata Post-Mint	Medium
M-23	Clone Registered Before Initial Buy - Revert Leaves Orphaned Registered Curve	Medium
M-24	Discontinuity in _calculatesqrtPriceX96 Causes Inconsistent Price Calculation	Medium
M-25	_mintNewPosition sets deadline = block.timestamp - no MEV protection on pool creation	Medium
M-26	LockPositionsNFT.tokenURI Is Caller-Dependent and Returns Different Metadata for the Same tokenId	Medium

ID	Title	Severity
L-01	Inherited burnFrom in BurnableToken.sol is not overridden, bypassing magnifiedDividendCorrections update and permanently bricking affected accounts for transfers	Low
L-02	Unchecked Transfer Return Values Across Multiple Contracts	Low
L-03	LockManager.distributeRewards Lacks Access Control - Anyone Can Inflate Reward Accounting	Low
L-04	LiquidityLocker Migration Leaves Stale Pool Enumeration State	Low
L-05	abi.decode Panic on Malformed Non-Error Oracle Response	Low
L-06	Unvalidated oracle winner wedges FeeAccount.performUpkeep until owner intervention, halting reward distribution and treasury sweeps	Low
L-07	Precision Loss in _calculatesqrtPriceX96 Due to Division-Before-Multiplication	Low
L-08	LiquidityMigrator._lockLiquidity never assigns locker named return value	Low
L-09	_transferEtherToFeeAccount() Sends Full Contract ETH Balance Instead of Withdrawn Amount	Low
L-10	Sub-Threshold Spare WETH Accumulates Indefinitely With No Recovery Path	Low
L-11	BondingCurve.receive() accepts bare ETH with no slippage protection	Low
L-12	_checkIsBalanceMoreThanGoal dead-zone leaves up to 0.0001 ETH of dust unrefunded	Low
L-13	BondingCurve._continuousBuy: diff variable computed but never used	Low
L-14	receiveDevReward Has No Access Control - ETH Can Be Sent by Anyone	Low
L-15	BondingCurvesStorage.withdrawDevRewards: No zero-amount check	Low
L-16	BondingCurvesStorage inherits the non-upgradeable ReentrancyGuard inside a UUPS proxy, omitting the base-contract storage gap required for safe upgrades.	Low

ID	Title	Severity
L-17	LockPositionsNFT::getOwnedTokenIds reverts instead of returning an empty array when skip exceeds the owner's token count.	Low
L-18	_calculateLockBoost Rounds Exact Two-Decimal Values Up Incorrectly	Low
L-19	LockPositionsNFT.burn Leaves Stale positions[tokenId] Storage Behind	Low
L-20	ERC721 Callback Reentrancy Can Desynchronize ownedTokenIds from Canonical Ownership	Low
L-21	Permissionless RewardResolver.performUpkeep allows LINK consumption and winner-selection timing manipulation	Low
L-22	Strict Equality on ETH Balance Can Be Broken by Forced Ether Injection	Low
L-23	ETH Sent Directly to Certain Contracts Is Permanently Locked	Low
L-24	User can create Bonding Curve for a token with improper metadata due to lack of validation	Low
L-25	Missing Zero-Address Validation for _feeAccount in BondingCurve.initialize()	Low
L-26	Division-by-Zero When totalSupply() == 0 in distributeDividends() and _withdrawDividend()	Low
L-27	Social Handles Cannot Be Cleared After Setting, Permanently Locking User Identity On-Chain	Low
L-28	_safeMint CEI Violation Opens Narrow Reentrancy Surface	Low
L-29	batchMint Partial Execution Provides No Signal to Caller	Low
L-30	BurnableToken.lock() requires approving the token contract itself (non-standard approval flow footgun)	Low
L-31	FeeAccount.performUpkeep treasury sweep includes bonding-curve trading fees, not just LP fees	Low
I-01	No Uniqueness Enforcement for Social Handles	Info
I-02	Multiple uint256 comparisons with <= 0	Info
I-03	Power.sol: Public mutable version string wastes storage	Info
I-04	FeeAccount.onERC721Received has unused parameters	Info
I-05	Withdrawing eth before checking upkeepNeeded wastes gas in FeeAccount::performUpkeep	Info
I-06	collectFeesAndDistribute NatSpec says the caller is rewarded, but no caller reward exists in the implementation	Info

Detailed Findings

C-01 - Permissionless Rogue Clone Attack: BondingCurve.initialize() Accepts Unvalidated Migrator and Factory Addresses, Enabling Total Fund Drain at Graduation

Critical

Description

BondingCurve.initialize() accepts caller-supplied liquidityMigrator and fee-account addresses without validating that the caller is the registered factory or that the supplied addresses are canonical. An attacker can clone the real BondingCurve implementation, initialize the clone through an attacker-controlled factory/storage chain, and configure a malicious migrator while preserving otherwise legitimate token, lock, and dividend behavior.

Impact

At graduation, the rogue curve approves the malicious migrator for the unsold token balance and forwards the entire ETH reserve to it. The migrator can drain the tokens and ETH, after which token ownership is renounced and recovery becomes impossible.

Recommendation

Restrict initialize() to a registered BondingCurveFactory, verify the migrator against canonical storage, and verify the fee account against the canonical migrator before writing state.

C-02 - LockManager: New lock positions can claim all past rewards and dividends

Critical

Description

LockManager uses a magnifiedRewardPerShare / magnifiedDividendsPerShare model to distribute rewards and dividends proportionally to lock shares. However, unlike BurnableToken (which uses magnifiedDividendCorrections mapping to track per-user corrections), LockManager has no correction term when new positions are created.

When a new lock position is minted, its collectedRewards[tokenId] and collected-Dividends[tokenId] start at 0. The collectable amount is:

accumulativeRewardOf is computed as (magnifiedRewardPerShare * lockShares) / MAGNITUDE, which includes all rewards ever distributed, not just rewards after the position was created. Since collectedRewards[tokenId] starts at 0, a new position can immediately claim a proportional share of ALL historical rewards.

Impact

Direct theft of rewards and dividends from legitimate lock holders. Every new lock position dilutes and steals from existing ones.

Recommendation

Add a correction term similar to BurnableToken.magnifiedDividendCorrections. When a new position is minted, record the current cumulative per-share values:

This sets the "already collected" baseline to the current accumulated amount, so the position can only claim rewards distributed after its creation.

C-03 - Double-Claim Token Drain via Reentrancy in claimRewards

Critical

Description

claimRewards has no nonReentrant modifier. Its execution sequence sends ETH to msg.sender via collectDividends before burning the position NFT. During this ETH transfer, a malicious recipient contract can reenter claimRewards. At reentry time, ownerOf(tokenId) still returns the attacker because the NFT has not yet been burned, and collectedRewards[tokenId] is still populated. The full claim executes a second time. LockManager holds tokens from all lockers in a single shared contract, so the reentrancy drains from the collective balance, not just the attacker's own allocation.

Impact

All tokens and ETH locked in LockManager are drainable. Every active position across every token is at risk. The shared pool means one attacker with one position can extract funds belonging to all other lockers.

Recommendation

Add nonReentrant to claimRewards. Apply CEI strictly: burn the NFT, zero collectedRewards[tokenId], and decrement totalLockShares before any external ETH or token transfers.

C-04 - Unguarded migrateLiquidity() Function Causes Loss of User Earnings (CVSS 9.5)

Critical

Description

The migrateLiquidity() function in LiquidityLocker is marked external override with no access control modifiers. This allows any address to trigger early migration of liquidity positions, which deletes the position from LiquidityLocker WITHOUT collecting accumulated fees first. This results in permanent loss of earned fees and orphaned earnings that cannot be recovered.

Impact

This vulnerability enables permanent loss of user earnings:

Why This Is CRITICAL:

Recommendation

Two critical issues must be fixed:

1. Collect fees BEFORE deleting position (prevents orphaning earned fees)
2. Add access control (prevents unauthorized calls)

Key Requirements:

- Collect fees via collectFeesAndDistribute() BEFORE position deletion
- Add onlyLiquidityMigrator access control modifier
- Test that no fees are orphaned during migration process

H-01 - Attacker can steal the proxies' assets

High

Description

The castr.fun protocol deploys all it's proxy contracts via a foundry deployment script DeployProject.s.sol. However foundry does not execute script txs atomically.

Anyone can frontrun the initialization of the proxies, become their owner and in future steal all the assets.

Due to the private mempool of Base, this attack is not easy to execute, but it is still possible by periodically executing the 'initialize() functions of the future deployed proxy addresses. Also it's cheap(given the cheap tx fees on base)

Impact

Theft of proxies' assets

H-02 - collectFeesAndDistribute missing return causes approve(address(0)) revert when lockManager is uninitialized

High

Description

When a token has accrued fees but no user has ever locked (so lockManager == address(0)), collectFeesAndDistribute is supposed to park the lock-share portion of fees at the BurnableToken address. The branch is missing a return:

OpenZeppelin's ERC20 approve reverts with ERC20InvalidSpender when the spender is address(0). So control flow always falls through to approve(address(0), ...) after the parking transfer and reverts.

Impact

- DoS on collectFeesAndDistribute for any graduated token during the window between migration and the first user lock() - a window that is always open and can last minutes to hours.
- Combined with H-01, the fee-collection path and the first-lock path mutually block each other in a wedge state that cannot be cleared by normal user action.
- Fees continue to accumulate at the Aerodrome pool uncollected; users who expected dividends from the lock-for-rewards system never receive them until the wedge is manually resolved.

Recommendation

Add return; immediately after the parking transfer, or restructure the branch as if/else, so the approve/distributeRewards calls execute only when lockManager != address(0).

H-03 - The Swap Protection Limit Is Calculated Using the Wrong Token and the Wrong Amount

High

Description

When the contract runs its automated reward swap, it is supposed to set a minimum acceptable output to protect itself from getting a bad deal. However, the code makes two mistakes that together make this protection completely useless.

First, it asks for a price quote between WETH and WETH, the same token on both sides, which makes no sense and produces a meaningless number. Second, it only uses part of the total funds when calculating that quote, instead of the full amount being swapped.

Because of this, an attacker can freely manipulate the swap price right before it executes and steal the platform's reward funds with no resistance from the contract.

The problem lives in the _rewardWinner() function inside FeeAccount.sol. To understand it, you need to know how the contract is set up first.

When a token graduates and a liquidity pool is created, LiquidityMigrator.sol registers that token in FeeAccount by calling addCompetingToken(). It passes WETH as the "other token" in the pair. This value gets saved permanently into the contract's storage like this:

Impact

- The platform's reward funds. ETH and WETH that were meant for token buy-and-burn and treasury distribution. can be stolen by any attacker.
- Because performUpkeep() is callable by anyone once the resolver reaches the right state, there is no access barrier. Any bot can do this.
- The losses are permanent. Each upkeep cycle is a new opportunity for the attacker.

Recommendation

The fix has two parts:

1. Use the correct token pair in the price quote

The quote should ask: *"how many winner tokens do I get for my WETH?"* not WETH for WETH. Change the call to:

2. Use the full swap amount in the quote

Replace amountInEth with amountInTotal (which is amountInEth + amountInWeth) so the minimum output is calculated for the actual amount being spent.

3. Add a safety check before proceeding

H-04 - Reward Swaps Use a Price That Can Be Manipulated in the Same Block

High

Description

Even if the token pair and amount issues from H1 were fixed, there is a second independent problem. The contract gets its price information from pool.slot0(), which gives the current instantaneous price at that exact moment. This type of price reading can be moved by an attacker in the same block, or even the same transaction, using borrowed funds. The attacker shifts the price, the contract trades at that wrong price, and the attacker immediately reverses everything and keeps the difference.

Inside _rewardWinner(), the contract reads the pool's current tick to calculate how many winner tokens it should receive:

slot0() returns a snapshot of the pool's price right now, in this block. It does not average anything over time. This means if someone moves the price just before this line runs, the contract will use that fake, temporary price as if it were real.

In Uniswap V3, moving the price of a pool does not require owning those tokens outright. An attacker can use a flashloan, borrowing a large amount for a single transaction for free - to push the price, trigger the FeeAccount swap at the wrong price, reverse their trade, repay the loan, and keep the profit. All of this can happen atomically in one block.

Impact

- Attackers can extract ETH/WETH from FeeAccount on every single upkeep cycle.
- Low-liquidity pools, the most common case for newly graduated tokens, are the easiest and cheapest to attack.

Recommendation

Replace slot0() with a TWAP oracle. Uniswap V3 has a built-in way to do this using OracleLibrary.consult():

A few extra considerations:

- Choose the right time window. A longer window (e.g. 30 minutes) is harder and more expensive for an attacker to manipulate, but it will lag behind real

price moves more. A shorter window is more responsive but cheaper to attack. 30 minutes is a common and reasonable starting point.

- Make sure the pool supports TWAP. The pool's observation cardinality must be large enough to cover your chosen time window. If the pool was just created, it may not have enough history. You should add a check for this and fall back gracefully or skip the swap if the TWAP is not available.

H-05 - performUpkeep/_rewardWinner Re-Includes LP-Fee WETH and Burns LP-Fee Winner Tokens

High

Description

The protocol has two distinct value domains inside FeeAccount:

1. LP-fee distribution domain (treasury/dividends + dev/lock rewards), handled by collectFeesAndDistribute.
2. Winner Token buyback-and-burn domain (Chainlink upkeep rewards), handled by performUpkeep and _rewardWinner.

performUpkeep explicitly tries to exclude reserved LP-fee WETH before rewarding winners:

However, _rewardWinner later recomputes:

At this point, wethBalance includes totalAccummulatedWethAmounts (reserved LP-fee WETH), so reserved fee funds are added back into swap budget and can be spent for winner buybacks.

There is a second inclusion bug on the token side: after swap, _rewardWinner burns the entire winner token balance held by FeeAccount:

If FeeAccount already holds winner tokens from LP-fee collection (e.g., pending/undistributed bucket), those LP-fee tokens are burned together with newly bought tokens.

Impact

1. LP-fee WETH intended for treasury/dividend distribution is consumed by upkeep buybacks.
2. Treasury and dividend recipients are underpaid due to silent accounting leakage.
3. Winner-token LP fees intended for dev/lock-holder distribution can be unintentionally burned.
4. Repeated upkeep cycles can continuously drain fee-distribution reserves into buybacks, causing persistent protocol-level loss of designated fee allocations.

H-06 - Permissionless pre-creation of the Slipstream pool permanently DoSes migration

High

Description

LiquidityMigrator always calls the Slipstream NonfungiblePositionManager.mint() entrypoint with a nonzero sqrtPriceX96.

In this vendored Slipstream implementation, mint() does not treat that value as a hint for an already-existing pool. Instead, when sqrtPriceX96 != 0, it first calls CLFactory.createPool(...). The factory reverts if the pool already exists.

Because Slipstream pool creation is permissionless, any third party can pre-create the exact token/WETH pool with tick spacing 2000 before a bonding curve reaches migration. Once that happens, every migration attempt for that token reverts before any liquidity is minted.

This invalidates the weaker "wrong initial price is accepted" framing for this repository. The actual effect is stronger: a hard per-token migration DoS.

Impact

- A third party can permanently block migration for a specific token.
- The curve cannot complete launch, because BondingCurve._checkIsMigrating() always calls into the reverting migrator path.
- This does not require the attacker to supply liquidity or win any race at migration time. Pre-creating the pool once is enough.

This is a real liveness failure in the launch pipeline, not merely a pricing discrepancy.

Recommendation

Handle existing pools explicitly instead of always attempting pool creation through mint().

Practical fixes:

- Query ICLFactory.getPool(...) before minting.
- If no pool exists, create it with the intended initial price.

- If a pool already exists, either revert with a protocol-specific error or only proceed after validating slot0() against the expected launch price within a tight tolerance.
- Split "create pool" and "add liquidity" into explicit branches instead of relying on mint() to do both.

As a defense-in-depth measure, if the protocol later decides to support pre-existing pools, it should also set meaningful amount0Min and amount1Min bounds.

H-07 - Dust Sent to the Token Contract Can Permanently Brick the First Lock Initialization

High

Description

The first call to BurnableToken.lock has a special initialization path that clones a new LockManager. Before any lock shares are minted, it checks whether the token contract itself already holds any token balance and, if so, immediately forwards that balance into LockManager.distributeRewards. The problem is that distributeRewards divides by totalLockShares, which is still zero during this first-time setup. As a result, any pre-existing token dust at the token contract address makes the very first lock-manager initialization revert.

Impact

Any user can permanently disable the locking subsystem for a token with a dust-sized transfer. This also blocks downstream flows that depend on successful lock-manager creation, including fee reward distribution paths that try to lock dev rewards.

H-08 - Stale Oracle Response Accepted - requestId Never Validated

High

Description

handleOracleFulfillment validates only that msg.sender == ROUTER_ADDRESS. It never checks requestId against s_lastRequestId. The error type RewardResolverUnexpectedRequestID exists in the contract but is never used. A delayed fulfillment from a prior cycle overwrites s_winner, triggers WinnerPicked with a stale address, and updates s_lastWinnerTimestamp - corrupting all future cycle query windows.

Impact

Incorrect winner selection across reward cycles. Corruption of temporal reward logic and the s_lastWinnerTimestamp window that governs which tokens are eligible for future cycles. Potential cascading inconsistency in all downstream reward distribution.

Recommendation

The error type is already defined. This is a one-line fix.

H-09 - Hardcoded Sepolia configurations in RewardResolver.sol cause total failure on Base mainnet deployment

High

Description

The protocol is going to be deployed on Base network. However, the RewardResolver.sol contract contains hardcoded constants specifically configured for the Ethereum Sepolia testnet.

Specifically, ROUTER_ADDRESS is hardcoded to the Sepolia Chainlink Functions router limit (0xb83E47C2bC239B3bf370bc41e1459A34b41238D0), and DON_ID is hardcoded to the bytes32 representation of the Sepolia DON.

When deployed to Base, this creates two fatal failures during the winner selection loop:

1. _sendFunctionRequest() will attempt to dispatch the request to a non-existent or incorrect Sepolia Router and DON on the Base network.
2. Even if a local Base Chainlink router somehow attempts to fulfill the callback via handleOracleFulfillment(), the transaction will permanently revert because it asserts if (msg.sender != ROUTER_ADDRESS). The Base router address will never match the hardcoded Sepolia address.

Impact

High. Permanent Denial of Service (DoS) of the protocol's winner selection and reward distribution system. Because RewardResolver will permanently fail to complete its Chainlink Functions callback, FeeAccount::performUpkeep() will never execute the actual asset payouts to winners, permanently freezing all generated rewards within the protocol.

H-10 - Missing Zero-Shares Guard in distributeDividends() and distributeRewards()

High

Description

Both functions perform share-distribution arithmetic with totalLockShares as denominator without an explicit zero-shares guard. When no active lock positions exist, totalLockShares == 0 and the division path reverts.

Impact

Dividend and reward distribution paths become non-executable while totalLock-Shares == 0.

Recommendation

Add explicit zero-shares guards before both denominator uses:

1. In distributeDividends(), return early when totalLockShares == 0.
2. In distributeRewards(), require totalLockShares > 0 before share update.
3. Keep arithmetic denominator in local variable after guard to enforce a single checked source.

H-11 - Dividend redistribution in case of failure leads to loss of funds.

High

Description

The protocol implements a dividend distribution mechanism using a magnified-DividendPerShare model. During token transfers, _beforeTokenTransfer triggers _withdrawDividend(from), attempting to send accumulated ETH dividends to caller through the following block,

If the call somehow fails (pool holds unvestedTokens and accumulates the dividend, but does not have a receive or fallback), the contract executes a fallback logic,

At first, it seems like the dividends from failed call are being automatically distributed to all other token holders through increased magnifiedDividendPerShare, letting caller forego it via updated withdrawnDividends[_owner], but upon furthur inspection, something far worse is happening (a Proof Of Concept is added for better understanding),

1. The failed dividend amount is redistributed across all holders, INCLUDING the original recipient (e.g., the pool)
2. Immediately after, the recipient's withdrawnDividends is set to its full accumulativeDividendOf, thus marking its entire

accrued dividend as already claimed.

Result

- Now, the protocol has magnifiedDividendPerShare * balanceOf(caller) which is equal to withdrawnDividends[caller]. This is

Impact

A portion of dividends become irrecoverable and accumulates in the contract over time.

Recommendation

Consider adding an emergency mechanism to withdrawn such funds, or only distribute dividends to non pool addresses. So the migrated tokens should be

subtracted from the totalSupply in distributeDividends calculation, and if, from == pool, the _withdrawDividend should not be called.

H-12 - virtualLiquidity Permanently Drifts Due to Sell Fee Accounting Error

High

Description

In _continuousSell, virtualLiquidity is decremented by the gross reimburse amount before the fee is deducted. The fee ETH is transferred out of the contract, but virtualLiquidity is reduced by the full pre-fee amount - meaning the virtual reserve understates the actual ETH balance by the cumulative sell fees across all sells.

Impact

The curve progressively underprices buys relative to the actual reserve, giving buyers more tokens than the bonding curve formula should issue. Over time the accounting diverges enough to permanently halt all buys via an underflow revert, effectively bricking the bonding curve for any token with high sell volume.

H-13 - Cloned bonding curves inherit constructor-initialized Power storage that is never initialized in clone instances

High

Description

The bonding curve pricing logic depends on a lookup table stored in Power.maxExpArray. That table is filled inside the Power constructor.

The problem is that live bonding curves are created through clones. Clone instances do not run the implementation constructor in their own storage.

This means a cloned bonding curve can start with an empty lookup table even though the pricing math expects that table to be present. That can lead to wrong pricing or unexpected reverts in the core buy and sell flow.

Recommendation

Do not rely on constructor-filled storage in contracts that will be used as clone implementations.

Move that setup into an initializer, or refactor the logic so it no longer depends on mutable constructor-filled storage.

H-14 - safeTransferFrom bypasses soulbound restrictions and desynchronizes LockPositionsNFT ownership tracking

High

Description

LockPositionsNFT overrides transferFrom to call _removeTokenId and _addTokenId. It does not override safeTransferFrom. The base ERC721 safeTransferFrom moves ownership via _update but never calls the tracking helpers. Every standard marketplace interaction - OpenSea, Blur, and all major platforms default to safeTransferFrom - permanently desynchronises ownedTokenIds.

State after a standard marketplace sale:

Because the base ERC721 safeTransferFrom path is not overridden consistently with transferFrom, marketplace-style safe transfers can bypass the intended soulbound restriction while also skipping the custom ownership-index book-keeping.

Impact

Buyers cannot discover their position via getOwnedTokenIds. Sellers retain ghost entries indefinitely. Every user who trades via any standard NFT interface is permanently affected - which covers the default interaction pattern for every major marketplace.

Lock-position NFTs can become transferable through safeTransferFrom despite the intended soulbound design, and auxiliary ownership tracking can permanently diverge from canonical ERC721 ownership.

Recommendation

Override both safeTransferFrom(address,address,uint256) and safeTransferFrom(address,address,uint256,bytes) with the same tracking logic. Centralise all tracking in an _update hook override to prevent future omissions.

H-15 - FeeAccount.onERC721Received accepts arbitrary LockPositionsNFT deposits, permanently stranding a perpetual share of every future reward and dividend distribution

High

Description

FeeAccount implements IERC721Receiver with a stub that accepts every NFT from every sender:

LockPositionsNFT.mint is called with _safeMint so that FeeAccount can legitimately receive a freshly-minted dev-cliff lock during collectFeesAndDistribute. This is the only legitimate flow that requires FeeAccount to accept an ERC721 callback. The stub does not distinguish that flow from any other source, so an attacker can route an *arbitrary* lock position into FeeAccount via the standard ERC721 safeTransfer.

The attack is a single permissionless sequence:

1. Attacker calls BurnableToken.lock(amount, durationInDays) for the target token. This is permissionless (anyone holding the dividend token can call it). The call mints the attacker a LockPositionsNFT position whose lockShares are added to LockManager.totalLockShares.
2. Attacker calls positionsNFT.safeTransfer(address(feeAccount), tokenId). The safeTransfer triggers onERC721Received on FeeAccount, which accepts unconditionally. FeeAccount is now ownerOf(tokenId).

Recommendation

The fix must allowlist senders rather than reverting outright (the dev-cliff mint path requires FeeAccount to accept its own LockManager's LockPositionsNFT). The cleanest implementation tracks the legitimate LockPositionsNFT per registered token and checks msg.sender against it.

Fix - sender allowlist on onERC721Received.

emitNewLockManagerAndNFT is the natural registration point because it is already called by BurnableToken.lock() the first time a LockManager clone is created (BurnableToken.sol#L437-L440), and it already has the onlyGraduatedTokens modifier. Adding the allowlist write there guarantees the legitimate LockPositionsNFT is registered before any safeTransfer from it can ever fire -

the dev-cliff mint cannot be invoked until *after* the LockManager has been created and registered.

H-16 - Fee Exemption Mechanism Critically Broken: Any Transferable NFT Enables Arbitrary Fee Bypass

High

Description

The protocol's fee exemption logic in _deductFee() checks if the recipient holds any NFT from the basedNFTs array, but does not enforce that these NFTs are non-transferable (soulbound). As a result, any standard ERC721 can be added to basedNFTs, allowing users to borrow or temporarily receive such an NFT, execute a buyToken or sellToken transaction, and immediately return the NFT - all within a single transaction. This completely nullifies the intended trading fee for any user with access to a transferable NFT.

Impact

This flaw allows attackers to bypass the 1% trading fee on any trade size, including multi-ETH transactions, with zero cost or risk. The attack is trivial to execute using existing NFT lending protocols or simple transfers, and does not require privileged access. The protocol's fee model is rendered unenforceable for all non-soulbound NFTs, resulting in unbounded loss of protocol revenue.

Recommendation

Enforce that all basedNFTs are non-transferable (soulbound) by requiring a strict interface or contract type check on addition. Additionally, iterate over all basedNFTs entries for fee exemption, not just index 0, and consider implementing a minimum holding period or snapshot mechanism to prevent flash-exemption attacks.

H-17 - Superlinear Vesting Exponent (1.3) Enables Sybil-Based Vesting Bypass via Pre-Graduation Wallet Splitting

High

Description

VestingCalculator.calculateVestingDuration() computes post-migration vesting as MIN + (share^1.3) * (MAX - MIN) where share = balance / totalVestedSupply. Because exponent 1.3 > 1, the function is superadditive: f(a+b) > f(a) + f(b). Splitting a single position into N wallets reduces each wallet's vesting to near-MIN_VESTING_PERIOD (1 day).

Pre-graduation, wallet-to-wallet transfers are allowed once unlockTimePreMigration expires (the vested modifier only blocks when unlockTimePreMigration > block.timestamp). The launchedOrOwnerTransfer modifier permits non-DEX transfers. _trackVestingPreMigration correctly propagates holdersVesting[to].amount += amount to each recipient.

Impact

High. A 10% holder reduces vesting from ~19 days to ~1 day (19x reduction) by splitting to 100 wallets. This defeats the protocol's stated "Non-Linear Holder Vesting" anti-dump mechanism. Post-graduation dumping causes direct loss to other holders.

Likelihood: High. Cost is ~100 L2 transfers on Base (~$0.10 total gas). No special permissions, flash loans, or market conditions required. Any rational whale is incentivized to do this.

Recommendation

Replace exponent 1.3 with a sublinear exponent (e.g., 0.7). With exponent < 1, sum(x_i^e) > (sum(x_i))^e, making splitting _increase_ aggregate vesting - Sybil-unprofitable. Alternatively, enforce a meaningful minimum floor (e.g., 7 days).

H-18 - Direct LockManager.lock() call bypasses post-migration vesting lock-floor in one transaction, letting any vested holder liquidate their entire balance during vesting

High

Description

The post-migration vesting lock-floor is a core protocol invariant: any holder whose balance is still vesting (holdersVesting[holder].amount > holdersVesting[holder].lockedAmount) must not be able to create a LockManager position whose unlock time precedes unlockTimePostMigration. The floor is enforced inside LockManager._checkVestingLockDurataion:

The assumption baked into this check is that unlockTimePostMigration is a non-zero future timestamp for every vested holder post-migration. It isn't. BurnableToken only fills the field in lazily, inside the vested modifier on transfer-/transferFrom:

setMigrated() does NOT anchor per-holder unlock times. Every vested holder that has not yet done a post-migration transfer therefore has unlockTimePost-Migration == 0 in storage - which is the default state for any holder that locks before moving tokens.

There are two entry points into the LockManager:

Impact

Every vested holder can independently bypass the post-migration vesting lock in one transaction plus a 1-day wait, and the bypass works permissionlessly with no prerequisites beyond "has not yet transferred post-migration" - which is the default state. The vesting schedule - a load-bearing invariant for the launch's distribution curve and for holder expectations about post-migration supply - becomes a fiction. Downstream consumers (getVestedAmount, dividend accounting, any UI or indexer that reads holdersVesting) continue to believe the balance is vesting-locked while the holder walks away with liquid tokens. In aggregate, the post-migration circulating supply curve is uncapped.

Recommendation

The check needs to read the value it actually cares about, not the stale storage field. Two viable fixes:

Option 1 (preferred): compute unlockTimePostMigration inside the check instead of reading storage.

calculateHolderVestingPostMigration already exists and is public view, so this is a single-line semantic fix. It returns the freshly-computed unlock time regardless of whether the storage field has been lazily initialized.

Option 2: eagerly anchor unlockTimePostMigration for every holder at migration time.

Have setMigrated() (or a subsequent loop / lazy write inside addLockedAmount) populate unlockTimePostMigration so the storage field is never 0 once migration is complete. This is more invasive because it requires iterating over all vested holders or adding a write inside the lock path.

H-19 - _transferEtherToFeeAccount() Will Always Revert Because Contract Has No receive() Function

High

Description

_transferEtherToFeeAccount() calls i_weth.withdraw(amount), which causes the WETH contract to push native ETH back to address(this) via a low-level call. For this to succeed, the receiving contract must implement receive() external payable or fallback() external payable. LiquidityMigrator defines neither. Every time a migration produces spare WETH exceeding the 0.1 ether threshold, the WETH withdraw() call reverts, rolling back the entire transaction - including pool creation, liquidity minting, and the fee account registration.

Impact

Any bonding curve migration that leaves more than 0.1 ether of spare WETH - a routine occurrence given position manager rounding - will permanently fail. The bonding curve cannot graduate, liquidity is never locked, and the protocol's core graduation mechanism is broken for all affected tokens.

H-20 - Soulbound Property Bypassed via safeTransferFrom

High

Description

The contract attempts to enforce non-transferability by overriding transferFrom, approve, and setApprovalForAll with unconditional reverts. However, it does not override either safeTransferFrom variant inherited from OpenZeppelin's ERC721.

In OpenZeppelin v5, used with Solidity 0.8.22, safeTransferFrom does not call the public transferFrom function. Instead, it calls the internal _safeTransfer function, which then routes directly to _transfer, bypassing the overridden public function entirely. The soulbound guarantee is therefore completely broken.

Attack Path

1. An attacker obtains or is gifted a soulbound NFT.
2. The attacker calls either safeTransferFrom(owner, attacker, tokenId) or safe-TransferFrom(owner, attacker, tokenId, bytes("")).
3. OpenZeppelin internally routes the call to _transfer, which has no soulbound check.
4. The transfer succeeds and the NFT is now held by a new address.
5. The token can be resold, delegated, or used to impersonate the original recipient.

Root Cause

Incomplete override of all ERC721 external transfer entry points. The over-ridden transferFrom function is bypassed by the internal call chain used by safeTransferFrom in OpenZeppelin v5.

Impact

- The core protocol invariant is violated: the supposedly soulbound NFT is fully transferable.
- Enables secondary markets, unauthorized resale, delegation, and identity spoofing.

- Any downstream privilege system that trusts soulbound ownership can be compromised because ownership is no longer guaranteed to remain bound to the original recipient.

Recommendation

Override both safeTransferFrom variants explicitly so that every external ERC721 transfer entry point reverts:

```
function safeTransferFrom(
    address,
    address,
    uint256
) public pure override {
    revert SoulboundNFT__NotTransferable();
}

function safeTransferFrom(
    address,
    address,
    uint256,
    bytes memory
) public pure override {
    revert SoulboundNFT__NotTransferable();
}
```

M-01 - BondingCurve drives king-of-the-casts and migration state transitions off address(this).balance, allowing forced-ETH griefing via selfdestruct

Medium

Description

BondingCurve has no internal reserve variable. Every state-threshold decision - whether the curve has reached its goal, whether to migrate, whether to crown the king-of-the-casts - reads address(this).balance directly:

startLiquidity in the crown check is snapshotted from virtualLiquidity - START_VIRTUAL_LIQUIDITY at the top of _buyToken() (L235), i.e. the "real ETH added" as tracked by the internal accounting. The crown condition then compares this virtual snapshot against the real on-chain balance - the two sides of the comparison use different ledgers.

receive() routes plain ETH transfers through buyToken(0, ""), so an ordinary .call{value: x}("") is treated as a normal buy and does not desynchronize the ledgers. However, selfdestruct transfers are not routed through receive() (and post-EIP-6780 they still transfer the balance, even when the selfdestructing contract was not created in the same tx). This gives an attacker a permissionless way to credit ETH to a curve without executing the buy path.

Recommendation

Introduce an internal reserve state variable that is updated exclusively through the sanctioned buy/sell paths, and use reserve - not address(this).balance - for every state-threshold comparison and for the value forwarded into migration:

Any unsolicited ETH that lands in the contract outside of reserve can then be handled explicitly - e.g. swept into the migrator's LP at migration time, or forwarded to the fee account - without allowing it to drive state transitions. This also removes the accounting asymmetry in _checkKingOfTheCasts where one side of the condition is virtual and the other is real, eliminating both the crown manipulation and forced migration vectors in one change.

M-02 - FeeAccount WETH accumulation double-counts, eventually reverting performUpkeep

Medium

Description

collectFeesAndDistribute tracks pending (non-treasury-cut) WETH per token in accummulatedWethAmounts[token] so it can be withdrawn and paid as dividends once the treasury cut becomes non-zero. The bookkeeping does wethAmount += accummulatedWethAmounts[token] (loading the old value into the local), and then, when treasuryAmount == 0, writes back the full new sum - including the old balance that was just added in:

Impact

- performUpkeep (the Chainlink Automation entry point) reverts forever once enough under-scale collections accumulate, freezing the winner-reward flow for all graduated tokens simultaneously.
- On the dividendAmount > 0 branch, weth.withdraw(dividendAmount) reverts because it tries to withdraw more WETH than the contract holds, or silently cross-subsidizes from WETH balance contributed by other tokens.
- Any low-volume token whose per-collection WETH amount is small enough to make treasury rounding hit zero is a trigger, and the damage is cumulative across the entire protocol.

Recommendation

Separate the *new* WETH amount from the *running* accumulator. Only add the new amount to accummulatedWethAmounts[token] and totalAccummulated-WethAmounts when treasuryAmount == 0:

M-03 - Reward Upkeep Spends WETH That Was Reserved for Per-Token Fee Distribution

Medium

Description

collectFeesAndDistribute accumulates small WETH fee amounts in accummulatedWethAmounts[token] and tracks the total in totalAccummulatedWethAmounts when the amount is too small to split into treasury and dividends. Those balances are meant to remain reserved until a later fee collection for the same token can distribute them properly. However, performUpkeep only excludes the reserved WETH from the initial withdraw step; _rewardWinner still uses the remaining on-contract WETH balance to fund reward buys. In practice, that means the upkeep path spends WETH that the accounting system still considers reserved for token holders.

Impact

Reserved per-token fee balances can be misallocated into the reward-buyback path, and the mismatch can later break reward upkeep or fee distribution until the accounting deficit is covered by new inflows.

M-04 - LockManager: Last Lock Redistribution Always Panics

Description

redistributeRewards is the path the fee account takes when burning an expired lock and spreading its tokens across remaining positions. The problem is in the order of operations - shares are subtracted first, then the result is immediately used as a denominator. When you're burning the last lock, that denominator hits zero and the whole thing panics.

Impact

Once a LockManager reaches its last lock, fee distribution is permanently bricked for that token. Every collectFeesAndDistribute call reverts. Treasury, dividends, dev rewards - all of it stops. The locked tokens and any rewards the last locker earned are stuck.

M-05 - Missing Slippage Protection During Liquidity Migration Enables Sandwich Attacks

Medium

Description

During liquidity migration, the protocol mints a position with both amount0Min and amount1Min set to 0. This allows attackers to manipulate price before execution, forcing the protocol to accept a bad ratio and burn leftover tokens.

When createPoolAndLockLiquidity is called during migration, it creates a new liquidity position without enforcing any minimum amount constraints:

js

amount0Min: 0,

amount1Min: 0,

This means the mint will succeed regardless of how unfavorable the price becomes before execution.

Because the function relies on the current pool price at execution time, any price movement between submission and execution directly affects how much liquidity is actually deposited.

Impact

- A portion of the protocol liquidity is permanently burned.
- The pool is initialized with less liquidity than intended.
- Early pool conditions become easier to manipulate due to reduced depth.

Recommendation

- Do not mint liquidity with zero minimums.

M-06 - BurnableToken.burn() under-clamps the vesting accumulator, letting any post-launch locker brick their own transfers with a 1-wei burn

Medium

Description

BurnableToken tracks the non-liquid portion of a holder's balance in two storage fields: holdersVesting[h].amount (call it V, the vesting snapshot) and holdersVesting[h].lockedAmount (call it L, the amount that has been committed through LockManager.lock). getVestedAmount assumes the invariant V >= L - it subtracts V - L unguarded:

L is written exactly once in the entire codebase - at addLockedAmount, which is append-only:

There is no path that decrements L - not on burn, not on claimRewards, not on redistributeRewards, not on lock expiry, not on removeLastKingOfTheCasts. Once a holder has locked anything through LockManager, their L is permanently non-zero for the rest of the contract's life.

V is mutated in three places: pre-launch seeding (_trackVestingPreMigration), post-migration calculateHolderVestingPostMigration views, and the burn() override. The burn() override decrements V but clamps the decrement against 0, not against L:

Impact

- Self-inflicted temporary freeze of the caller's balance (Medium, DoS / no-profit griefing). A 1-wei burn after a legitimate lock wedges the caller's transfer / transferFrom for the remainder of unlockTimePostMigration - up to 365 days for whales, ~1-30 days for typical holders. The frozen balance is exactly the holder's post-lock residual plus any subsequent inflows - the protocol's "burn-for-dividend-share" mechanic advertises this exact interaction as normal usage.

Recommendation

Clamp burn()'s decrement of V against L, not against 0. The burnable portion of V is V - L (the slack between the vesting snapshot and what has already been committed through LockManager), and burning must never be allowed to push V below that floor:

This preserves the V >= L invariant by construction and does not require any changes to getVestedAmount, _checkVestedAmount, or the vested modifier. The fix applies symmetrically to the M-07 burnFrom fix: once burnFrom is overridden to call the same inline logic as burn, both entry points share this same clamp.

M-07 - BondingCurve._checkIsMigrating passes stale tokensForLiquidity to setMigrated after _manageSpareTokens burn

Medium

Description

_checkIsMigrating captures tokensForLiquidity before migration, then passes it to token.setMigrated:

LiquidityMigrator._mintNewPosition may not use the full tokensForLiquidity amount because of tick alignment and sqrt-price imbalance. The leftover is passed to _manageSpareTokens, which burns the surplus at LiquidityMigrator.sol:310. After the burn, totalSupply() is less than the value captured before migration, but setMigrated(tokensForLiquidity) is called with the pre-burn number.

Impact

- totalVestedSupply is set below the true vested balance, which skews every holder's VestingCalculator denominator and thus their vesting duration / unlock schedule.
- Concrete direction and magnitude depend on setMigrated's formula; worst case users experience unlock times that diverge from design intent, or VestingCalculator reverts from an underflow condition during later vesting math.

Recommendation

Use the return values from createPoolAndLockLiquidity (the actual amount0/amount1 consumed by the mint), or re-read token.balanceOf(address(this)) after _manageSpareTokens has burned the leftover, and pass that refreshed amount to setMigrated.

M-08 - kingsOfTheCasts Array Corruption When currentKingOfTheCasts Migrates

Medium

Description

When a bonding curve that currently holds the currentKingOfTheCasts title completes and migrates, _checkKingOfTheCasts executes three sequential if blocks within the same call. The interaction will execute Condition 2 which forces the Condition 3 and causes removeLastKingOfTheCasts to operate on a stale assumption about which array index holds lastKingOfTheCasts, permanently corrupting the kingsOfTheCasts history array.

updateKingOfTheCasts (called by Condition 2) when isCrowned = true:

removeLastKingOfTheCasts (called by Condition 3) invariant assumption:

But condition 2 calls updateKingOfTheCasts with isCrowned = true, which shifts lastKingOfTheCasts to point at the migrating(current king) curve - the *last* array element at index length-1 - without touching the array. When Condition 3 immediately calls removeLastKingOfTheCasts, the function deletes kingsOfTheCasts[length-2] (the previous king, B) instead of kingsOfTheCasts[length-1] (the migrating curve, C).

Impact

Given the pre-state kingsOfTheCasts = [A, B, C], lastKingOfTheCasts = B, currentKingOfTheCasts = C, and now C migrates (isOnCurve = false):

Condition 2 executes: updateKingOfTheCasts(B, isCrowned=true)

- lastKingOfTheCasts = C (the migrating curve, at array index 2)
- currentKingOfTheCasts = B
- Array unchanged: [A, B, C]

Condition 3 executes: removeLastKingOfTheCasts()

- Targets kingsOfTheCasts[length-2] = B (index 1) - wrong; real lastKing C is at index 2
- Overwrites B with C: [A, C, C] -> pops -> [A, C]
- Sets lastKingOfTheCasts = kingsOfTheCasts[0] = A

Corrupted final state:

M-09 - basedNFTs[0] Only Ever Checked - Multi-NFT Fee Waiver Support Is Broken

Medium

Description

_deductFee in BondingCurve checks only basedNFTs(0). The basedNFTs array can grow unboundedly via addBasedNFT but all entries beyond index 0 are permanently ignored. There is no removeBasedNFT function. If the first NFT becomes defunct or its contract is compromised, fee waivers cannot be recovered without a contract upgrade.

Impact

Fee waiver functionality is effectively limited to a single NFT contract permanently. If that contract becomes invalid, fee waivers break entirely. If it becomes malicious, fee waivers can be exploited. All additional NFTs added via addBasedNFT are wasted storage.

Recommendation

Iterate all entries in _deductFee, or replace the array with a mapping and implement a removeBasedNFT function for lifecycle management.

M-10 - Bonding Curve Permanent DoS Due to Underflow in BondingCurve::_extendHolderTransfersLock

Medium

Description

The bonding curve uses two different accounting values to track progress, and they fall out of sync near the end of the curve:

1. address(this).balance - the gross ETH held by the contract. Used by _checkIsBalanceMoreThanGoal to decide when to set isOnCurve = false.
2. virtualLiquidity - START_VIRTUAL_LIQUIDITY - the cumulative net ETH deposited after fees. Used as startLiquidity and passed to _extendHolderTransfersLock.

The curve completes (isOnCurve = false) when address(this).balance >= maxBuy (~4.5454 ETH for a 1% fee). But virtualLiquidity only tracks net deposits. Fees are immediately sent out via _deductFee. This creates a window where cumulative net deposits exceed BONDING_CURVE_GOAL (4.5 ETH) while address(this).balance has not yet reached maxBuy.

In this window, any buy reverts because _extendHolderTransfersLock computes:

When balance (i.e. startLiquidity) exceeds BONDING_CURVE_GOAL, curveProgress > SCALE, and Solidity 0.8's checked arithmetic causes the subtraction to revert. The early return guard (if (!isOnCurve) return) does not help because isOnCurve is still true in this window.

Impact

Once cumulative net deposits exceed 4.5 ETH without triggering isOnCurve = false, every subsequent buy reverts. The curve is permanently frozen - it can never reach maxBuy to trigger migration to a liquidity pool. All tokens and ETH remain locked on the bonding curve with no recovery path.

M-11 - VestingCalculator Reverts on Zero totalVestedSupply

Medium

Description

If totalVestedSupply == 0 - reachable if setMigrated is called with liquidityAmount >= totalSupply() - every call to calculateVestingDuration reverts. The vested modifier in BurnableToken calls this function on every transfer post-migration. All token transfers become permanently impossible.

M-12 - checkUpkeep Does Not Validate ETH Balance, Leading to Wasted LINK

Medium

Description

In the FeeAccount and RewardResolver contracts, checkUpkeep triggers upkeep execution without confirming that the FeeAccount actually has ETH to distribute. This causes the protocol to spend LINK on Chainlink Functions requests that end up doing nothing.

The upkeep flow is designed to run periodically (e.g., every 24 hours) and distribute ETH rewards. However, the trigger condition in checkUpkeep only checks:

solidity

upkeepNeeded = timeHasPassed && isOpen;

It does not verify whether the FeeAccount has any ETH balance. As a result, even when there are no funds to distribute, the upkeep is still triggered.

Impact

- LINK subscription funds are drained
- Upkeep execution may stop entirely once LINK runs out
- Reward distribution becomes unreliable and dependent on manual intervention

Recommendation

- Ensure checkUpkeep verifies that there are funds available before triggering upkeep.

Stephen - 09/04/2026 07:35

2ND REPORT:

M-13 - Fee Percentages in FeeAccount Not Validated Against 100% Cap

Medium

Description

setDevRewardPercent, setTreasuryRewardPercent, setBasePercent, and setAdditivePercent have no cross-validation. With defaults (basePercent=500, additivePercent=100) and 3 winners, total allocation is 500 + 600 + 700 = 1800/1000 = 180%. Each winner's percentage is applied to the current remaining balance sequentially, so the first winner takes their full cut, later winners receive progressively less. Silent over-commitment with no revert.

Impact

Silent over-allocation silently under-distributes to later winners. In pathological configurations, the distribution loop reverts mid-execution, leaving some winners rewarded and others not - creating an inconsistent reward state that cannot be retroactively corrected.

Recommendation

Add a combined validation function called inside every setter:

M-14 - Incorrect reward calculation causes inversed reward calculation for winners in performUpkeep

Medium

Description

The performUpkeep() natspec mentions the following:

> @dev This is the function that Chainlink nodes will call to reward the picked top tokens that bonded. Each token's pool receives a portion of the total rewards. This is calculated as basePercent + additivePercent * multiplier.

> The multiplier is determined by the place in reverse order (if top 3 - 3th place has multiplier 1, 2nd place has multiplier 2, 1st place has multiplier 3).

According to the natspec the rewards multiplier are determined as follows, assuming top-3 winners:

- 3rd place —> multiplier = 1 (least reward)
- 2nd place —> multiplier = 2
- 1st place —> multiplier = 3 (most reward)

That means the best (1st place) should receive the highest multiplier (3), and the worst (3rd place) the lowest (1).

This is the expected behavior of the function but the calculation for this logic is as follows:

According to this code block above, where multiplier = i (loop index):

If winners is ordered from best to worst (typical), then:

- i = 0 (1st place) -> smallest multiplier ()
- i = 1 (2nd place) -> middle multiplier
- i = 2 (3rd place) -> largest multiplier

This is the exact opposite of the natspec and the described behavior.

M-15 - collectRewardsAndDividends Has No ownerOf Guard

Medium

Description

The public wrapper has no ownerOf modifier. While inner functions enforce ownership on their own execution, any address can call the wrapper to force reward accounting state transitions on any tokenId - moving pending rewards from the formula into collectedRewards without the owner's knowledge or consent.

Impact

Any caller can force reward accounting state changes on any position without the owner's consent. While direct fund theft is blocked by inner ownerOf checks, the forced state snapshot can be used to time-manipulate reward accounting against a position holder.

Recommendation

Add ownerOf(tokenId) modifier directly to collectRewardsAndDividends.

M-16 - State Machine Deadlock in CALCULATING State in RewardResolver.sol

Medium

Description

The contract can become permanently stuck in the CALCULATING state if the Chainlink Functions oracle fails to respond or if the oracle response is lost. Once performUpkeep sets the state to CALCULATING, only a successful oracle response can transition the state to WAITING. There is no timeout mechanism or emergency recovery function to reset the state if the oracle becomes unresponsive. The only way to recover is through the setStateOpen function, but this requires the feeAccount to be available and aware of the issue.

Impact

The contract could become permanently inoperable if oracle responses fail, preventing future reward distributions. This creates a single point of failure that could halt the entire reward system indefinitely.

Recommendation

Implement a timeout mechanism to automatically reset the state after a reasonable period:

BUG-1: redistributeRewards divides by zero on last position

M-17 - FeeAccount.collectFeesAndDistribute Lets the Freshly Minted Dev Lock Share in the Same Reward Distribution

Medium

Description

In FeeAccount.collectFeesAndDistribute, the dev portion of the collected token fees is locked first, and only after that the remaining lockRewardAmount is distributed to lock holders:

burnableToken.lock() mints a new lock position for the dev and increases LockManager.totalLockShares immediately. Then LockManager.distributeRewards() updates magnifiedRewardPerShare using that already-increased totalLockShares.

As a result, the newly created dev lock participates in the same-cycle locker reward distribution that is supposed to be distributed to existing lockers. This means the dev does not only receive the intended locked principal (devRewardAmount), but also receives an immediate share of lockRewardAmount.

Impact

This dilutes the reward share of pre-existing lockers on every fee collection where a dev lock is minted successfully.

The issue is low severity because the extra rewards are bounded by the configured dev allocation and reward timing, but it still breaks the intended accounting split:

- dev should receive only the locked dev allocation for that cycle
- existing lockers should receive the full lockRewardAmount

Instead, part of lockRewardAmount is redirected to the dev's freshly created lock.

Assume:

- existing lockers currently have 100 total lock shares
- tokenBalance = 100
- devRewardPercent = 20%
- devRewardsCliff = 30 days

Then:

- devRewardAmount = 20

Recommendation

Ensure the dev lock does not participate in the same distribution round. For example:

- distribute lockRewardAmount before minting the dev lock, or
- snapshot the pre-dev-lock totalLockShares and distribute against that value, or
- defer eligibility of newly minted locks until the next reward epoch

M-18 - lock() First Caller Silently Receives Undocumented Boost - First-Mover Advantage

Medium

Description

The first address to call lock() on any token triggers lazy LockManager initialization and receives isBoosted = true - a permanent 5% additional lock shares for the entire lock duration. All subsequent callers receive isBoosted = false. This is completely undocumented, creates a race condition on every token launch, and directly contradicts the fair launch positioning of the protocol.

Impact

Every token launch has a race condition where a bot can capture a permanent economic advantage over all other lockers. The fair launch guarantee is violated for the locking system on every token.

Recommendation

Remove the isBoosted parameter from lockFor, or apply the initialization boost to the protocol treasury (distributing it as initial rewards to the contract itself) rather than to the first external caller.

M-19 - Deterministic Rarity Calculation Is Predictable and Gameable

Medium

Description

The rarity check uses a fully on-chain, deterministic formula:

Both inputs - tokenId and poolAddress - are known before the mint transaction is confirmed. Any actor can compute the exact rarity outcome for any given tokenId off-chain before minting.

Impact

- Rare status is gameable by any actor who can read chain state
- Breaks fairness assumptions for all legitimate participants
- Undermines the perceived value and distribution integrity of rare tokens

Recommendation

- Use Chainlink VRF to inject verifiable randomness at mint time
- Alternatively, implement a commit-reveal scheme where the rarity seed is committed before the token ID is known
- At minimum, incorporate block.prevrandao as an additional entropy input (weaker but better than nothing)

M-20 - Unconstrained Handle Input on Mint Enables JSON Injection and Storage Bloat

Medium

Description

The mint() function accepts arbitrary-length strings for all three handle fields (twitter_xHandle, farcasterHandle, telegramHandle) with no length validation whatsoever. The update functions (updateTwitterXHandle, etc.) enforce strict maximum lengths, but those constraints are entirely absent on the mint path.

These raw strings are later concatenated directly into a JSON string inside tokenURI with no escaping:

This creates two distinct issues: a JSON injection vector in the metadata output, and an unbounded storage write on every mint.

Impact

- Metadata JSON corruption and false trait injection
- Unbounded storage writes per token; potential gas denial-of-service on tokenURI for tokens with very large handles
- Inconsistency between data written at mint and data permitted by update functions creates permanently irrecorrectable state (handles can only be updated to valid lengths, but the overlong original remains if update is never called)

Recommendation

Extract handle validation into a shared internal function and call it from both mint() and the update functions:

Apply the same pattern for Farcaster and Telegram handles. Call these validators inside mint() before writing to tokenMetadata.

Additionally, consider HTML/JSON-safe character allowlisting (alphanumeric + underscore) to fully eliminate injection surface.

M-21 - tokenURI Does Not Validate Token Existence

Medium

Description

The overridden tokenURI function does not verify that the queried tokenId corresponds to a minted token. It reads directly from tokenMetadata[tokenId], which returns a zero-valued struct for any unminted ID.

Impact

- Potential for marketplace display of phantom tokens
- Incorrect supply signals to off-chain indexers

Recommendation

Add an existence check at the top of tokenURI:

M-22 - Owner Can Arbitrarily Mutate Global Metadata Post-Mint

Medium

Description

The setImage() and setAnimation() functions allow the contract owner to replace the image and animation URLs for all tokens simultaneously at any time after deployment. Recipients of the NFT have no protection against their token's visual representation being changed without notice.

Impact

- Post-mint rug vector on visual metadata for all token holders simultaneously
- No transparency: changes are silent (no event)
- Erodes trust for recipients who hold the token as a credential or badge

Recommendation

- Emit events on every metadata change:
- Consider making image and animation immutable if they are not intended to change post-deployment
- Alternatively, enforce a timelock before changes take effect, giving token holders time to react

M-23 - Clone Registered Before Initial Buy - Revert Leaves Orphaned Registered Curve

Medium

Description

In _createBondingCurve, i_bondingCurvesStorage.addBondingCurve(bondingCurveClone) is called before buyTokenForDev. If the initial buy reverts - for example, dust ETH causes the fee to round to zero - the storage registration is not rolled back. The orphaned curve remains permanently registered as a valid bonding curve but has no liquidity and no recovery path.

Impact

Registered curves that failed their initial buy appear valid to any contract or indexer reading bondingCurves[address]. Any future logic that trusts this mapping without additional validation can be misled. Dev reward flows and storage-level checks can interact incorrectly with these orphaned entries.

Recommendation

Move addBondingCurve and emitNewBondingCurveCreated to after buyTokenForDev completes successfully.

M-24 - Discontinuity in _calculatesqrtPriceX96 Causes Inconsistent Price Calculation

Medium

Description

The _calculatesqrtPriceX96 function applies a piecewise formula gated by:

The two branches use different scaling logic. At the exact boundary (amount1 == 18 ether), the two formulas produce materially different output values. There is no continuity guarantee - the price output function has a hard discontinuity at this threshold.

Impact

- Identical economic states produce different prices depending on which side of the threshold they fall on
- Creates a predictable arbitrage surface exploitable in every block
- May result in loss of funds for LPs positioned near the 18 ether boundary
- Undermines price oracle reliability for any downstream consumer

Recommendation

- Replace with a unified continuous formula valid across the full input range
- If a piecewise approach is necessary for precision reasons, enforce continuity at the boundary: verify that f(18 ether) yields the same value under both branches
- Add invariant tests that assert continuity across the threshold with fuzz inputs in the range [17.9 ether, 18.1 ether]

M-25 - _mintNewPosition sets deadline = block.timestamp - no MEV protection on pool creation

Medium

Description

When minting the liquidity position during migration, the MintParams struct sets the deadline to block.timestamp:

A deadline of block.timestamp is equivalent to no deadline at all - validators can include the transaction in any future block and block.timestamp will always be valid at inclusion time. This means the migration transaction can be delayed, reordered, or sandwiched by MEV bots with no time-based protection.

Impact

Every token migration to Aerodrome CL is vulnerable to MEV. Liquidity can be added at a manipulated price with no protection. This affects token prices at launch for every project using the protocol.

Recommendation

Pass the deadline as an external parameter from the caller or use an offchain-computed deadline:

M-26 - LockPositionsNFT.tokenURI Is Caller-Dependent and Returns Different Metadata for the Same tokenId

Medium

Description

LockPositionsNFT.tokenURI is not deterministic for a given NFT. While metadata for an ERC721 token is expected to depend only on token state, this implementation injects msg.sender into the SVG generation path:

The downstream descriptor uses that address as minterAddress when constructing the SVG, which affects displayed fields and color generation. As a result, the exact metadata returned for the same tokenId changes depending on who calls tokenURI.

This breaks the core expectation that an NFT's metadata is stable and token-centric. Wallets, marketplaces, indexers, and direct callers can each observe different metadata for the same position NFT even though no onchain token state changed.

Impact

Wrong metadata information.

The same tokenId can display different metadata and image output for different callers of tokenURI, which can mislead users, break marketplace/indexer consistency, and make the NFT appear to represent different ownership or visual traits depending on who queried it.

L-01 - Inherited burnFrom in BurnableToken.sol is not overridden, bypassing magnifiedDividendCorrections update and permanently bricking affected accounts for transfers

Low

Description

BurnableToken inherits ERC20BurnableUpgradeable, which exposes a public burnFrom function. While burn() is overridden to update magnifiedDividendCorrections and vesting state, burnFrom is never overridden - it calls _burn() directly, skipping all custom accounting:

Compare with the overridden burn() which correctly maintains dividend corrections:

When burnFrom is called, balanceOf(account) decreases but magnifiedDividendCorrections[account] is not adjusted. This breaks the invariant that the raw accumulative value (magnifiedDividendPerShare * balanceOf + corrections) is preserved through balance changes.

The corruption manifests in two ways depending on the account's state:

Failure mode 1 - accumulativeDividendOf reverts (negative raw value):

For accounts that received tokens via transfer, magnifiedDividendCorrections is negative (decreased by magnifiedDividendPerShare * amount on receipt). After burnFrom reduces the balance without compensating the correction, the raw sum magnifiedDividendPerShare * new_balance + corrections goes negative, causing _toUint256Safe to revert inside accumulativeDividendOf.

Failure mode 2 - withdrawableDividendOf underflows:

Impact

Any account that calls burnFrom (or has an approved spender call it) post-migration will have corrupted dividend accounting. If the account has previously collected dividends or received tokens via transfer, their remaining tokens become permanently untransferable until sufficient new dividends are distributed - which may never happen. The function also bypasses launch restrictions and vesting tracking.

Recommendation

Override burnFrom to include the same correction and vesting logic as burn():

Alternatively, disable burnFrom entirely if it is not intended to be part of the token's interface:

L-02 - Unchecked Transfer Return Values Across Multiple Contracts

Low

Description

Multiple transfer() and transferFrom() calls across four contracts do not check the return value, relying on the called token to revert on failure. For the currently deployed token types (BurnableToken and WETH, both of which revert on failure), this pattern is safe. The risk materializes if dividendToken is ever set to a non-standard ERC20 that returns false instead of reverting - in which case failed transfers would be silently ignored, corrupting accounting.

Impact

Currently low risk given the deployed token types. Future risk if dividendToken changes to a token that returns false on failure. Silent transfer failures would corrupt lock share accounting or cause reward claims to appear successful when funds were not actually moved.

Recommendation

Use OpenZeppelin's SafeERC20.safeTransfer() and safeTransferFrom() wrappers at all transfer call sites. This is particularly important for dividendToken transfers in LockManager, where the token type is configurable.

L-03 - LockManager.distributeRewards Lacks Access Control - Anyone Can Inflate Reward Accounting

Low

Description

LockManager.distributeRewards() is guarded only by a positive(amount) modifier with no onlyFeeAccount or equivalent access restriction. Any external caller who holds dividendToken and approves the LockManager can invoke it directly, injecting an arbitrary quantity of tokens into the reward pool and inflating magnifiedRewardPerShare. This is notable because the sibling function redistributeRewards() is correctly guarded by onlyFeeAccount, creating an asymmetry that may have misled reviewers into assuming distributeRewards is similarly restricted.

When combined with H-02 (Missing Reward Correction Initialization), a carefully timed injection followed by a lock() call allows an attacker to claim a disproportionate share of the injected tokens as retroactive rewards at the expense of existing lockers.

Impact

An external caller can donate tokens to the LockManager reward pool, artificially inflating magnifiedRewardPerShare. When combined with H-02, this enables reward accounting manipulation at the expense of existing lockers.

Recommendation

Add the onlyFeeAccount modifier to distributeRewards(), consistent with the restriction already present on redistributeRewards().

L-04 - LiquidityLocker Migration Leaves Stale Pool Enumeration State

Low

Description

LiquidityLocker tracks locked liquidity using three coupled state representations:

- lockedPositionsCount
- pools[]
- lockedPositions[pool]

On migration, the contract clears lockedPositions[pool] and decrements lockedPositionsCount, but it never removes the migrated pool from pools[]. That leaves stale pools at old indexes while still-live pools may sit beyond the range implied by lockedPositionsCount.

Recommendation

Keep the array, count, and mapping in sync on migration. Valid fixes include:

- removing the migrated pool from pools[]
- or deprecating lockedPositionsCount as an enumeration bound and exposing an explicit live-pool iterator/filter

L-05 - abi.decode Panic on Malformed Non-Error Oracle Response

Low

Description

_fulfillRequest() guards the abi.decode call with an err.length == 0 check, correctly skipping decoding when the DON returns an explicit error. However, if the DON returns a response with err.length == 0 (success flag) but the response bytes do not conform to the expected uint256 ABI encoding (e.g., response.length < 32), Solidity's abi.decode panics. This path is reachable if the Chainlink JavaScript source is malformed, returns a truncated value, or if Chainlink infrastructure truncates the response. Since the source code is mutable, the surface exists. A panic leaves s_state permanently stuck in CALCULATING with no recovery path short of an upgrade.

Impact

If triggered, the RewardResolver is permanently stuck in the CALCULATING state and no further reward cycles can run. Recovery requires a contract upgrade.

Recommendation

Add a response.length >= 32 guard before abi.decode. On malformed response, treat the result as a no-winner epoch (winner = address(0)) and proceed to State.WAITING rather than panicking.

L-06 - Unvalidated oracle winner wedges FeeAccount.performUpkeep until owner intervention, halting reward distribution and treasury sweeps

Low

Description

FeeAccount.performUpkeep takes whatever address RewardResolver returned from its Chainlink Functions response and uses it directly as a storage key into competingTokens, with no membership check:

_rewardWinner immediately dereferences the struct through an external call:

competingTokens is only populated during graduation via LiquidityMigrator.addLiquidity -> FeeAccount.addCompetingToken, and there is no removal path (the mapping is append-only). Any winner address that is not an already-graduated, registered token resolves to a default-initialized TokenRefferences, where pool == address(0). The subsequent slot0() call is a high-level external call to address(0); Solidity's compiler-inserted extcodesize check reverts the entire transaction.

Because setStateOpen() executes at L390 *inside* the same transaction as the reverting loop, a single bad winner creates a rollback wedge:

1. RewardResolver._fulfillRequest stores the DON response verbatim - winner = address(uint160(decodedResponse)) - and sets s_state = WAITING.
2. Chainlink Automation calls FeeAccount.performUpkeep. checkUpkeep passes because the resolver is in WAITING.
3. Line 390 sets the resolver back to OPEN.

Impact

Once the resolver returns any nonzero, unregistered address, the permissionless reward pipeline is halted until the owner intervenes:

- RewardResolver stays in WAITING, so no new Chainlink Functions request can be issued.
- The treasury.call{value: address(this).balance} sweep at L404 never executes - ETH collected from bonding fees sits idle in FeeAccount until recovery.
- The reward buyback-and-burn loop stops, pausing a core protocol economic

mechanism.

Reachability of the invalid winner is entirely within the privileged / off-chain surface:

Recommendation

Validate the winner against competingTokens and skip-rather-than-revert on any invalid entry, so a single bad oracle response cannot strand the state machine:

In addition, harden the upkeep against partial failures so one bad entry (or any other unexpected revert inside _rewardWinner, e.g. from the swap router) cannot roll back the resolver reopen:

1. Move rewardResolver.setStateOpen() after the loop, or wrap each _rewardWinner call in a try/catch so the resolver transition is never undone by a single failing winner.
2. Add an owner-gated emergency reset on RewardResolver (e.g. adminReset-ToOpen()), so an operator can unwedge the state machine without a full proxy upgrade in the event any future _rewardWinner revert mode is discovered.

L-07 - Precision Loss in _calculatesqrtPriceX96 Due to Division-Before-Multiplication

Low

Description

LiquidityMigrator::_calculatesqrtPriceX96 uses a branching approach to avoid uint256 overflow when computing the Uniswap V3 sqrtPriceX96. The correct formula is:

For amount1 > 18 ether, the function splits the multiplication to avoid overflow:

This introduces division-before-multiplication - a classic Solidity anti-pattern. The EVM truncates integer division toward zero, permanently discarding the remainder from (amount1 * Q96) / amount0 before the second Q96 factor is applied. The actual computation becomes:

This is not equivalent to the correct formula. The lost precision is amplified when the result is then multiplied by another Q96, since the discarded fractional bits are in the high-magnitude range.

Impact

Concentrated liquidity AMMs like Uniswap V3 derive tick boundaries directly from sqrtPriceX96. A truncated price value will cause:

1. Incorrect tick alignment at pool initialisation. The initial price set in the pool will be slightly lower than the true ratio, permanently mispricing the seeded liquidity position.
2. Systematic rounding in favour of one token. Every pool created via createPoolAndLockLiquidity will have a deterministic, exploitable price deviation. Arbitrageurs can extract value from the pool immediately upon migration by trading against the miscalculated initial price.

Recommendation

Use FullMath.mulDiv, which performs (a * b) / c using 512-bit intermediate arithmetic with no overflow and no premature truncation. This eliminates both the overflow risk and the precision loss in a single, branchless operation:

This is the approach used in Uniswap V3's own periphery contracts and is the production standard for this calculation.

L-08 - LiquidityMigrator._lockLiquidity never assigns locker named return value

Low

Description

locker is never assigned. The event and return value are always address(0).

Impact

- LiquidityLocked events always emit address(0), making off-chain indexers useless for this field.
- Latent correctness risk: routing behavior depends on an accidental address(0).

Recommendation

Have i_liquidityLocker.lock(...) return the locker address and assign it to locker before emitting the event. Alternatively, read the locker back via a getter on the LiquidityLocker keyed by poolAddress/tokenId and assign that.

L-09 - _transferEtherToFeeAccount() Sends Full Contract ETH Balance Instead of Withdrawn Amount

Low

Description

_transferEtherToFeeAccount() first calls i_weth.withdraw(i_weth.balanceOf(address(this))) to convert WETH to ETH, then immediately sends address(this).balance - the contract's entire native ETH balance - to the fee account. If the contract holds any ETH from a prior source (failed partial call, accidental direct send, or ETH from a previous msg.value that was not fully consumed), that ETH is silently swept into the fee account regardless of its origin.

Impact

Low under normal operation (contract should hold no residual ETH between calls), but any ETH that accumulates - from failed calls, direct sends, or edge-case partial operations - is silently drained to the fee account without accounting. This is also a latent reentrancy surface: if _transferEtherToFeeAccount() is ever callable in a context where a reentrancy path exists between the WETH withdraw and the ETH send, balance manipulation becomes possible.

L-10 - Sub-Threshold Spare WETH Accumulates Indefinitely With No Recovery Path

Low

Description

_manageSpareTokens() sends leftover WETH to the fee account only when i_weth.balanceOf(address(this)) > 0.1 ether. When the spare amount is at or below this threshold, the function resets the allowance to zero and returns - leaving the WETH stranded in the contract. There is no sweep function, no accumulation-and-send logic, and no owner rescue path. Every migration that produces less than 0.1 ether of spare WETH contributes to a permanently growing stranded balance.

Impact

Cumulative permanent loss of protocol ETH. Across a high-volume deployment, the stranded amount scales linearly with migration count. No individual migration is catastrophic, but the aggregate loss is material and unrecoverable without a contract upgrade.

L-11 - BondingCurve.receive() accepts bare ETH with no slippage protection

Low

Description

Any ETH sent directly to the bonding curve triggers a buy with minAmount = 0.

Impact

MEV extractable from any ETH accidentally forwarded to the curve. No fund lock, but every such accidental send is sandwichable.

Recommendation

Either remove receive() entirely (forcing callers to use buyToken(minAmount, data) explicitly), or revert in receive() with a clear message so users cannot accidentally buy without slippage protection.

L-12 - _checkIsBalanceMoreThanGoal dead-zone leaves up to 0.0001 ETH of dust unrefunded

Low

Description

If the overflow above maxBuy is less than 0.0001 ether (1e14 wei), the refund is swallowed rather than returned to the buyer. The dust is added to the curve reserve and effectively redistributed pro-rata to other holders.

Impact

Minor per-user value loss at the migration boundary (<= 1e14 wei per buyer). The dust is not locked - it is silently redistributed to remaining holders, which is not the documented behavior.

Recommendation

Remove the 0.0001 ether dead-zone and always refund the exact excess: refundAmount = totalAmount - maxBuy;.

L-13 - BondingCurve._continuousBuy: diff variable computed but never used

Low

Description

An expensive calculation is performed but the result is never stored or used:

calculateContinuousSellReturn calls the Bancor calculateSaleReturn which involves complex power/log calculations. This wastes significant gas on every buy transaction.

Impact

Gas waste on every buy. No security impact.

Recommendation

Remove the unused calculation.

L-14 - receiveDevReward Has No Access Control - ETH Can Be Sent by Anyone

Low

Description

BondingCurvesStorage.receiveDevReward() is marked external payable with no access restriction. Any address can call it and credit ETH to any dev address in devRewards. The intended caller is a BondingCurve contract distributing fees to the token's developer. Without an access check restricting callers to registered BondingCurve contracts, arbitrary ETH can be associated with any dev address in the mapping.

Impact

No user funds are at direct risk. The primary concern is inflated devRewards accounting and unrelated entries in the mapping. An attacker crediting their own address and immediately withdrawing is a no-loss self-send.

Recommendation

Add an access control check restricting callers to registered BondingCurve contracts, using a registry mapping or the onlyCurve pattern used elsewhere in the codebase. At minimum, emit an event with msg.sender to detect unauthorized calls off-chain.

L-15 - BondingCurvesStorage.withdrawDevRewards: No zero-amount check

Low

Description

If reward == 0, the function still makes a low-level call with 0 value and could emit a misleading DevRewardsWithdrawn event with amount 0.

Impact

Gas waste and misleading events for zero-reward withdrawals.

Recommendation

Add if (reward == 0) return; at the start.

L-16 - BondingCurvesStorage inherits the non-upgradeable ReentrancyGuard inside a UUPS proxy, omitting the base-contract storage gap required for safe upgrades.

Low

Description

BondingCurvesStorage is deployed behind an ERC1967 (UUPS) proxy yet imports ReentrancyGuard from @openzeppelin/contracts/utils/ReentrancyGuard.sol - the non-upgradeable variant - instead of ReentrancyGuardUpgradeable from the upgradeable package.

The same import is used in LockManager, though that contract is deployed as an immutable EIP-1167 clone and therefore carries no upgrade risk.

Impact

The reentrancy guard operates correctly and no funds are at immediate risk. The missing storage gap means that if the dependency is updated to a version of ReentrancyGuard that introduces a second state variable, deploying a new implementation of BondingCurvesStorage would shift every subsequent storage slot by one position, silently corrupting all mapped state (bondingCurves, devRewards, lockManagerImplementation, etc.). BondingCurvesStorage owns its own gap[100] to absorb future additions to its own variables, but this does not compensate for the missing gap in the base contract.

Recommendation

Replace the non-upgradeable import with the upgradeable variant and initialise it in initialize:

Apply the same change to LockManager for consistency, even though clones are immutable and carry no upgrade risk.

L-17 - LockPositionsNFT::getOwnedTokenIds reverts instead of returning an empty array when skip exceeds the owner's token count.

Low

Description

getOwnedTokenIds applies a bounds-correction to limit before guarding against an out-of-range skip. If skip > ids.length, the expression ids.length - skip underflows and reverts (Solidity 0.8.x checked arithmetic). The skip >= ids.length early-return guard that would have prevented this is placed after the underflowing subtraction, so it never fires.

Impact

Any caller paginating with a skip value beyond the owner's token count receives a revert instead of an empty array. This breaks pagination logic in off-chain indexers and front-ends that probe the end of the list, and will cause any on-chain integration that iterates positions to revert unexpectedly.

Recommendation

Move the skip guard before the limit correction:

L-18 - _calculateLockBoost Rounds Exact Two-Decimal Values Up Incorrectly

Low

Description

LockPositionsNFT::_calculateLockBoost uses 1e18 internal precision and then compresses the result into a value with 2 implied decimals. However, the custom rounding logic over-rounds certain values that already sit exactly on two decimal places.

The function currently derives the 2-digit decimal component with:

This does not correctly inspect only the next decimal digit for rounding. Instead, it evaluates a wider slice of the fixed-point remainder, which causes exact values such as 123.45 to be rounded upward.

Example:

- let the internal fixed-point value be 12345e16, representing exactly 123.45
- then:
- wholePart = 123
- decimals = 45e16
- decimalPart = 45
- ((decimals * 1000) / SCALE) % 100 = 450 % 100 = 50, so the branch increments decimalPart

So an exact 123.45 value is converted into 123.46, which is mathematically incorrect.

L-19 - LockPositionsNFT.burn Leaves Stale positions[tokenId] Storage Behind

Low

Description

When a lock position NFT is burned, the contract removes the token id from the owner's tracked list and burns the ERC721 token, but it does not delete the corresponding positions[tokenId] entry.

As a result, position metadata for burned NFTs remains in storage indefinitely even though the token no longer exists. This creates stale state that can be consumed by custom read paths and increases the chance of confusing offchain integrations that assume a burned position's data is cleared together with the NFT.

L-20 - ERC721 Callback Reentrancy Can Desynchronize ownedTokenIds from Canonical Ownership

Low

Description

LockPositionsNFT keeps a secondary ownership index in ownedTokenIds, but updates that index only after _safeMint / _safeTransfer complete. Both ERC721 flows invoke onERC721Received on contract recipients before ownedTokenIds is synchronized. A receiver contract can reenter transferFrom(...) during the callback and move the NFT again while the auxiliary index still reflects the pre-finalized state.

This produces a consistent pattern:

1. Canonical ERC721 ownership (ownerOf) changes during the callback-driven reentrant transfer.
2. The outer function resumes and performs its own delayed ownedTokenIds update.
3. The same token ID becomes present in multiple ownedTokenIds[...] buckets even though ownerOf(tokenId) has only one owner.

Impact

1. The canonical ERC721 owner remains correct.
2. Authorization gates that depend on ownerOf(...) remain intact.
3. No direct theft or reward-claim bypass was observed from this desynchronization alone.
4. The confirmed impact is that getOwnedTokenIds(...) can return stale / duplicated logical ownership, which can mislead UIs, indexers, or any future protocol logic that relies on this auxiliary view as a source of truth.

Recommendation

Bring ownedTokenIds bookkeeping into the same atomic state transition as the transfer itself.

1. Update auxiliary ownership state before or within the transfer path rather than after _safeMint / _safeTransfer returns.
2. Alternatively, eliminate the custom index and derive ownership from canonical ERC721 state.
3. If the custom index must remain, add reentrancy protection or restructure the flow so callback-based reentry cannot observe partially updated ownership bookkeeping.

L-21 - Permissionless RewardResolver.performUpkeep allows LINK consumption and winner-selection timing manipulation

Low

Description

RewardResolver.performUpkeep is externally callable without an access-control restriction. Any address can trigger a valid upkeep as soon as checkUpkeep returns true, causing the contract to send a Chainlink Functions request and spend subscription LINK. Because the caller controls the exact transaction timing, the same permissionless entry point also determines when the winner-selection request is initiated.

Impact

Third parties can consume LINK whenever upkeep becomes eligible and can strategically choose the execution time of winner selection. This creates avoidable oracle-cost exposure and timing manipulation of the reward cycle.

Recommendation

Restrict performUpkeep to the intended automation forwarder or keeper, or otherwise enforce an authorized-caller policy while retaining checkUpkeep validation.

L-22 - Strict Equality on ETH Balance Can Be Broken by Forced Ether Injection

Low

Description

FeeAccount uses strict equality checks against address(this).balance in two places. Strict equality on contract balances is unreliable because any contract can send ETH via selfdestruct, bypassing the receive function and making the actual balance differ from any internally tracked value.

Impact

An attacker who forces a small amount of ETH into the contract can cause the strict equality checks to permanently fail, disrupting the logic that depends on those conditions within the upkeep and fee collection flow.

L-23 - ETH Sent Directly to Certain Contracts Is Permanently Locked

Low

Description

BondingCurveFactory, FactoryManager, and RewardResolver each have payable functions or can receive ETH, but none expose any function to withdraw ETH sent directly to the contract address. ETH delivered outside of the expected function calls cannot be recovered.

Impact

Any ETH sent directly to these addresses is permanently locked. The risk is limited to accidental transfers or forced ETH injection and does not affect normal protocol operation.

L-24 - User can create Bonding Curve for a token with improper metadata due to lack of validation

Low

Description

According to the natspec of BondingCurveFactory::createBondingCurveForthe metadata param is described as:

@param metadata The metadata of the token, which can include additional information such as description, image URL, etc. It is abi-encoded and includes (in this order): required - description, image URL; optional - Telegram, Twitter, Farcaster and Website.

Required fields are description and image URL for the token being created, but the metadata input is not validated to ensure this required fields are present. Allowing a user to create a token with empty (bytes(0)) metadata bypassing the protocol intended required fields.

Mitiagation

Add a metadata validation to check for empty inputs, or add a minimal length that the protocol expects for the metadata

L-25 - Missing Zero-Address Validation for _feeAccount in BondingCurve.initialize()

Low

Description

BondingCurve.initialize() persists _feeAccount without validating address(0). The same state variable is later used as the direct recipient for protocol fee transfers in _deductFee().

Impact

Low-severity misconfiguration risk. Incorrect initialization of _feeAccount can irreversibly burn protocol fees and break expected fee-accounting behavior.

Recommendation

Add an explicit zero-address guard in initialize():

Additionally, validate upstream wiring in factory/migrator deployment scripts to ensure getFeeAccount() can never resolve to zero.

L-26 - Division-by-Zero When totalSupply() == 0 in distributeDividends() and _withdrawDividend()

Low

Description

Both paths divide by totalSupply() without checking for zero supply. Under Solidity 0.8.x semantics, integer division by zero triggers Panic(0x12) and reverts.

Impact

Low-severity liveness failure in dividend accounting paths once totalSupply() == 0. This is an edge-state issue, but it creates persistent revert behavior for affected dividend operations.

Recommendation

Add explicit supply guards before both divisions:

L-27 - Social Handles Cannot Be Cleared After Setting, Permanently Locking User Identity On-Chain

Low

Description

SoulboundNFT.mint() accepts empty strings for all three social handles, explicitly treating empty as the "unset" state. However, updateTwitterXHandle(), updateFarcasterHandle(), and updateTelegramHandle() all reject empty strings unconditionally. Once a user sets a handle via an update function, or if the protocol later adds a handle to an existing token, there is no on-chain path to remove it. The state transition is permanently one-directional: unset -> set, never set -> unset.

Impact

Users who set a social handle cannot remove it. Since the NFT is soulbound and cannot be transferred, there is no escape path - the social identity association is permanently locked to the token. This is a privacy concern and a violation of reasonable user expectations for updateable metadata fields.

L-28 - _safeMint CEI Violation Opens Narrow Reentrancy Surface

Low

Description

In mint(), _safeMint is called before tokenMetadata[tokenId] is written. If the recipient is a contract, _safeMint triggers an external call to onERC721Received before the metadata state is committed.

This is a classic Checks-Effects-Interactions violation.

Impact

Low in current form due to onlyOwner on mint(). Would become High if access controls are ever relaxed.

Recommendation

Move metadata write and event emission before _safeMint:

L-29 - batchMint Partial Execution Provides No Signal to Caller

Low

Description

batchMint exits early when gas falls below MIN_GAS_PER_MINT, returning the index of the last successfully minted token. However, the return value is identical whether the batch completed fully or stopped early - the caller cannot distinguish between the two outcomes without additional off-chain inspection.

Impact

- Silent partial mints mislead off-chain coordination systems
- Recipients may be incorrectly assumed to hold a token they never received

Recommendation

Emit an event on early exit to make partial completion detectable:

L-30 - BurnableToken.lock(): Requires User to Approve the Token Contract Itself

Low

Description

The lock function in BurnableToken uses an external self-call to transferFrom when moving a user's tokens into the token contract for locking:

```
whitelistedVesting[address(this)] = true;
this.transferFrom(msg.sender, address(this), amount);
whitelistedVesting[address(this)] = false;
```

Because this.transferFrom is an external call, msg.sender inside transferFrom becomes address(this) rather than the original user. As a result, transferFrom performs an allowance check against the token contract itself, requiring the user to first approve address(token) to spend their tokens before calling lock().

The required interaction therefore follows a non-standard approval flow: the user must call approve(address(token), amount) before invoking lock(). Users or integrations that are unaware of this requirement will encounter an unexpected revert when attempting to lock tokens directly.

Reviewer Note

This finding was downgraded from Medium to Low because the external this.transferFrom call is a deliberate design choice rather than an implementation bug.

The external call is required to trigger the vested and preTransfer modifiers applied to transferFrom. These modifiers perform the protocol's vesting checks and dividend-correction accounting. Replacing the external call directly with an internal _transfer would bypass this logic and could therefore violate the intended accounting and vesting behavior.

The remaining concern is primarily a UX and integration issue, which can be mitigated by frontends explicitly requesting the required approval before calling lock().

Impact

Users who attempt to call lock() without first approving the BurnableToken contract itself will encounter an unexpected revert. This approval pattern is unusual because users normally approve a separate spender contract rather than the ERC20 token contract itself.

Frontends, integrations, and users interacting directly with the contract must therefore be aware of this non-standard prerequisite and execute the approval transaction before locking tokens.

Recommendation

Document the required approval flow clearly and ensure that official frontends request approve(address(token), amount) before allowing a user to call lock().

Alternatively, the contract could explicitly execute the required vesting checks and dividend-correction accounting inside lock() and then use an internal _transfer. However, this approach must preserve all behavior currently enforced by the vested and preTransfer modifiers and should not simply replace this.transferFrom with _transfer without reproducing those checks.

L-31 - FeeAccount.performUpkeep: Remaining ETH Sent to Treasury Includes Bonding Curve Fees

Low

Description

After rewarding the selected winner inside FeeAccount.performUpkeep, the contract sends its entire remaining ETH balance to the treasury:

```
(bool success, ) = treasury.call{
    value: address(this).balance
}("");
```

At this point, address(this).balance can contain ETH originating from multiple sources rather than only unused funds associated with the current upkeep operation.

The remaining balance may include:

1. WETH converted to ETH during performUpkeep and intended for the reward flow.
2. Bonding curve trading fees received directly as raw ETH through _deductFee and subsequently forwarded to FeeAccount through _transferEther.
3. ETH received through collectDividends fallback paths.

Because the treasury transfer uses the contract's entire ETH balance, all bonding curve fees held by FeeAccount are swept to the treasury together with the remaining upkeep funds.

Reviewer Note

This finding was downgraded from Medium to Low because the observed behavior is likely intentional rather than an accounting bug.

Bonding curve trading fees arrive directly as raw ETH in FeeAccount.receive(). The reviewed codebase does not contain a separate mechanism for distributing these fees to token holders, developers, or lockers. The collectFeesAndDistribute flow handles LP fees generated after graduation, but does not account for bonding curve trading fees.

This indicates that bonding curve fees are likely intended to represent protocol revenue that ultimately belongs to the treasury. If those fees were intended to be distributed among other participants, an explicit accounting and tracking mechanism would be required, which is currently absent.

Impact

Bonding curve trading fees accumulated as ETH in FeeAccount are swept to the treasury when performUpkeep completes.

This is likely consistent with the intended protocol design and therefore does not currently represent a direct loss of funds. However, the intended treatment of bonding curve fees should be confirmed with the protocol team to ensure that treasury-only allocation is expected.

Recommendation

Confirm with the protocol team that bonding curve trading fees are intentionally designated as treasury revenue.

If this behavior is intended, document it explicitly so that the accounting model and fee destinations are clear to users, integrators, and future developers.

If bonding curve fees are instead intended to be distributed to holders, developers, lockers, or other recipients, introduce separate accounting for those fees rather than relying on the aggregate address(this).balance. This would prevent them from being unintentionally included in the treasury sweep performed by performUpkeep.

I-01 - No Uniqueness Enforcement for Social Handles

Info

Description

Multiple tokens can be minted with identical Twitter, Farcaster, or Telegram handles. There is no mapping or registry to enforce that a given handle is associated with only one token.

Impact

- Identity impersonation: multiple tokens can claim the same social identity
- Downstream systems that resolve handle -> token may behave non-deterministically

Recommendation

Optionally add uniqueness mappings:

Check and update on every mint and handle update. Whether this is desirable depends on protocol design intent.

I-02 - Multiple uint256 comparisons with <= 0

Info

Description

Several checks use <= 0 for uint256 values which can never be negative:

These are equivalent to == 0 but slightly misleading. Solidity 0.8.x uint256 cannot be negative.

Recommendation

Use == 0 for clarity.

I-03 - Power.sol: Public mutable version string wastes storage

Info

Description

This is a mutable state variable but is never changed. Since Power is inherited by BancorBondingCurve which is inherited by every BondingCurve clone, each clone stores this string in its own storage.

Recommendation

Change to string public constant version = "0.3.1"; to save storage per clone.

I-04 - FeeAccount.onERC721Received has unused parameters

Info

Description

All four parameters are unused, producing compiler warnings on every build.

Impact

Compiler-warning noise. No functional consequence. (The function also could be pure.)

Recommendation

Remove the parameter names (keep only the types) and mark the function pure.

I-05 - Withdrawing eth before checking upkeepNeeded wastes gas in FeeAccount::performUpkeep

Info

Description

FeeAccount::performUpkeep calls checkUpkeep to see if an upKeep is need but first withdraws eth before ensuring that this value is the desire.

This calling sender will spend more gas yet the transaction will revert instead of this extra spending being prevented by first checking the value

Recommendation

Consider checking the value before doing any thing else

I-06 - collectFeesAndDistribute NatSpec says the caller is rewarded, but no caller reward exists in the implementation

Info

Description

The NatSpec for FeeAccount::collectFeesAndDistribute states:

> Collects fees from the token's pool, distributes them, and rewards the caller.

However, the implementation does not contain any logic that rewards msg.sender.

The function only distributes value to these parties:

- treasury, via weth.transfer(address(treasury), treasuryAmount)
- burnableToken.dev(), via a lock NFT when devRewardAmount > 0
- token holders, via burnableToken.distributeDividends{value: dividendAmount}()
- lock holders, via lockManager.distributeRewards(lockRewardAmount) or redistributeRewards(lockId)

There is no branch that transfers tokens, ETH, WETH, or any other reward to the caller who triggers collectFeesAndDistribute.

Impact

This is a documentation / integrator-expectation issue rather than a direct fund-loss bug. Off-chain automation, keepers, or integrators may rely on the NatSpec and assume the function includes an execution incentive for the caller, when in reality it does not.

That mismatch can lead to:

- incorrect operational assumptions about how fee collection is incentivized
- failed keeper or bot economics if operators expect reimbursement or profit
- misleading protocol documentation for users and auditors

Recommendation

Align the documentation with the implementation, or add explicit caller-compensation logic if rewarding the executor is intended.

If no caller reward is intended, update the NatSpec to say the function only collects fees and distributes them to the configured recipients.

Disclaimers

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource, and expertise-bound effort in which we aim to identify as many issues as possible.

About Kann Audits

Kann Audits is a top-tier Web3 security audit company, trusted by leading projects and providing comphrensive audits and expert guidance to secure the most critical blockchain protocols.

Kann Audits provides services such as manual auditing, AI audits using the Kann AI Labs tool, and incident response.

To learn more, visit https://kannaudits.com
To view our audit portfolio, visit https://github.com/KannAudits
To book an audit, message https://t.me/KannAudits